

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Industry Problem Solving Task (Medium)
Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.	BL4 – Analyze	5
2	A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.	BL4 – Analyze	5
3	A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.	BL4 – Analyze	5
4	A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.	BL4 – Analyze	5
5	Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.	BL3 – Apply	5

6	An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.	BL4 – Analyze	5
7	For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.	BL4 – Analyze	5
8	Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.	BL4 – Analyze	5
9	A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.	BL4 – Analyze	5
10	Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.	BL4 – Analyze	5

Code of Conduct:

I Sandra Sudevan(192424422) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Sandra Sudevan

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of	25%	Applies advanced and relevant	Applies appropriate concepts	Limited application of concepts	Incorrect or no

Engineering Concepts		concepts effectively to solve the problem	with minor errors		application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

1. E-commerce — 10 Million Product Records

Requirement: Store and retrieve 10 million product records efficiently.

Recommended File Organization: Sorted File + B+ Tree Index

Reason:

- Product records can be sorted by ProductID.
- B+ Tree provides fast searching.
- Supports both exact searches and range queries such as price ranges.
- Sequential access is efficient for reports.

Cost Analysis:

Assume each record = 100 bytes.

- Total data = $10,000,000 \times 100 = 1 \text{ GB}$
- With 4 KB blocks: approximately **244,141 blocks**
- Sequential scan = **$O(N)$**
- B+ Tree search = approximately **$O(\log N)$**
- Insert/delete costs more than heap because records may need reorganization, but B+ Tree maintains efficient access.

Conclusion: B+ Tree indexed sorted organization is suitable because the huge dataset requires fast retrieval and range searching.

2. Banking — Transactions by Account Number and Date

Requirement: Quickly retrieve transactions using account number and date range.

Indexing Strategy:

```
CREATE INDEX idx_account_date
ON Transactions(AccountNo, TransactionDate);
```

Index Type: Composite B+ Tree Index

Example query:

```
SELECT *
FROM Transactions
WHERE AccountNo = 10101
AND TransactionDate BETWEEN '2026-01-01' AND '2026-01-31';
```

Why B+ Tree?

- Fast account-number lookup.
- Efficient date-range searching.
- Composite index supports both conditions.
- Leaf nodes maintain sorted order.

Conclusion: A composite B+ Tree index on (AccountNo, TransactionDate) is the best choice.

3. Healthcare — PatientID and Doctor Name

Requirement: Access patient records using both PatientID and DoctorName.

Multi-Level Indexing Design:

Primary Index:

PatientID → Patient Record

Secondary Index:

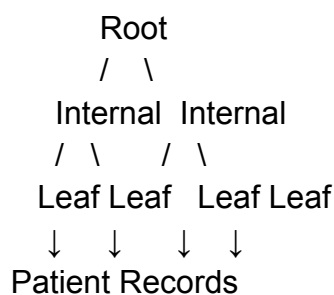
DoctorName → PatientID → Patient Record

Use **B+ Tree indexes**:

```
CREATE UNIQUE INDEX idx_patient  
ON Patient(PatientID);
```

```
CREATE INDEX idx_doctor  
ON Patient(DoctorName);
```

Multi-level structure:



Advantages:

- PatientID gives direct fast lookup.
- DoctorName supports retrieval of all patients belonging to a doctor.
- Multi-level indexing reduces disk accesses.

Conclusion: A primary B+ Tree index on PatientID and secondary B+ Tree index on DoctorName provides efficient access through both attributes.

4. Logistics — B-Tree vs B+ Tree

Requirement: Frequent insertion and deletion of shipment records.

Feature	B-Tree	B+ Tree
Search	$O(\log N)$	$O(\log N)$
Insert	$O(\log N)$	$O(\log N)$
Delete	$O(\log N)$	$O(\log N)$
Range search	Good	Excellent
Sequential access	Good	Excellent

Leaf nodes

Records may exist at all levels

Records stored at leaves

Recommendation: B+ Tree

Reason:

- Shipment systems commonly need range searches such as:
 - shipments between two dates
 - shipments within a location range
 - shipments by delivery status
- Leaf nodes are linked, making sequential and range access efficient.
- Internal nodes contain only keys, allowing more keys per block.

Conclusion: B+ Tree is more suitable for a dynamic logistics database because it provides efficient search, insertion, deletion, and range queries.

5. HR System — 5,000 Employees

Requirement: Store and retrieve 5,000 employee records using hashing.

Static Hashing

EmployeeID → Hash Function → Bucket

Example:

$h(\text{EmployeeID}) = \text{EmployeeID} \% 100$

Advantages:

- Simple.
- Fast average lookup.
- Easy to implement.

Disadvantage:

- Fixed number of buckets.
- Overflow can occur when employee records increase.

Extendible Hashing

Directory

↓

Bucket 1

Bucket 2

Bucket 3

Bucket 4

Advantages:

- Buckets dynamically split.
- Database can grow without major restructuring.
- Reduces overflow problems.

Feature	Static Hashing	Extendible Hashing
Initial size	Fixed	Dynamic
Search	O(1) average	O(1) average
Growth	Difficult	Easy
Overflow	More likely	Lower
Complexity	Low	Higher

Conclusion: For 5,000 employees with possible future growth, **extendible hashing** is preferred.

6. Airline Reservation — Flight Number + Date Range

Requirement:

- Point query by FlightNo
- Range query by FlightDate

Recommended: Composite B+ Tree Index

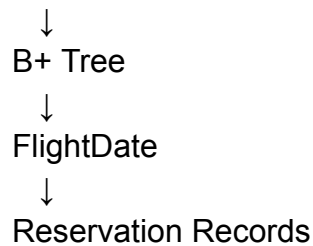
```
CREATE INDEX idx_flight_date  
ON Reservation(FlightNo, FlightDate);
```

Example:

```
SELECT *  
FROM Reservation  
WHERE FlightNo = 'AI202'  
AND FlightDate BETWEEN '2026-08-01' AND '2026-08-31';
```

Structure:

FlightNo



Advantages:

- Fast exact search for flight number.
- Efficient date-range search.
- Sorted leaf nodes support sequential retrieval.

Conclusion: A composite B+ Tree on (FlightNo, FlightDate) is suitable.

7. University — Department and CGPA

Requirement: Fast queries based on department and CGPA range.

File Organization: Heap File

Student records can be stored in heap organization because students are frequently inserted.

Secondary Index:

```
CREATE INDEX idx_dept_cgpa  
ON Student(Department, CGPA);
```

Example:

```
SELECT *  
FROM Student  
WHERE Department = 'CSE'  
AND CGPA BETWEEN 8.0 AND 9.5;
```

Why composite B+ Tree?

- First searches department.
- Then efficiently searches the CGPA range.
- Supports range queries.

Conclusion: Heap file + secondary composite B+ Tree index provides efficient insertion and retrieval.

8. Disk Block Utilization

Given:

- Records = **100,000**
- Record size = **50 bytes**
- Block size = **4 KB = 4096 bytes**

Records per Block

$$4096 / 50 = 81.92 \Rightarrow \frac{4096}{50} = 81.92$$

So approximately **81 records per block**.

Number of Blocks

$$100000 / 81 \approx 1235 \text{ blocks} \Rightarrow \frac{100000}{81} \approx 1235 \text{ blocks}$$

Heap File

Records are stored wherever space is available.

- Block utilization $\approx$ **99%**
- Approximately **1235 blocks**
- Excellent insertion performance.

Sorted File

Records are maintained in sorted order.

- Approximately **1235 blocks**
- Good sequential access.
- Insertions may require shifting/reorganization.

Hashed File

Records are distributed using a hash function.

- Approximately **1235 data blocks**, plus possible overflow blocks.
- Good equality search.
- Poor for range queries.

Organization	Blocks	Search	Insert	Range Query
Heap	≈ 1235	$O(N)$	Excellent	Poor
Sorted	≈ 1235	$O(\log N)$	Moderate/Poor	Excellent

Hashed	≈1235 + overflow	O(1) avg.	Excellent	Poor
--------	---------------------	--------------	-----------	------

Conclusion: Heap provides simple insertion, Sorted provides efficient ordered/range access, and Hash provides fastest equality search.

9. Social Media — 500 Million Posts

Attributes:

- user_id
- timestamp
- hashtag

Recommended Composite Indexing Strategy:

```
CREATE INDEX idx_user_time
ON Posts(user_id, timestamp);
```

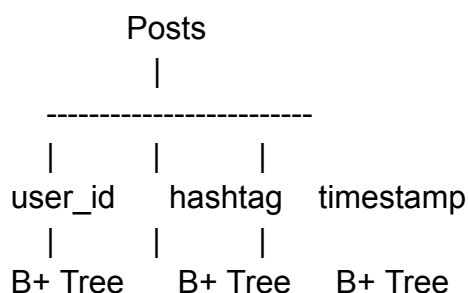
For hashtags:

```
CREATE INDEX idx_hashtag_time
ON Posts(hashtag, timestamp);
```

For time-based queries:

```
CREATE INDEX idx_timestamp
ON Posts(timestamp);
```

Strategy:



- user_id + timestamp → user's recent posts.
- hashtag + timestamp → posts using a hashtag within a period.
- Timestamp index → recent/trending posts.
- For 500 million records, indexes should be carefully maintained because indexes themselves consume significant storage.

Conclusion: Multiple targeted B+ Tree indexes are appropriate, with partitioning by time also useful at this scale.

10. Supermarket — 1 Million Daily Transactions

Compare Heap, Sorted, and Hashed File Organizations.

Operation	Heap File	Sorted File	Hashed File
Insert	$O(1)$ average	$O(N)$	$O(1)$ average
Search	$O(N)$	$O(\log N)$	$O(1)$ average
Delete	$O(N)$	$O(N)$	$O(1)$ average
Range Search	Poor	Excellent	Poor
Sequential Scan	Excellent	Excellent	Poor
Maintenance	Easy	High	Moderate

Heap File

Best when:

- Transactions are inserted continuously.
- Sequential processing is required.
- Exact search is not frequent.

Sorted File

Best when:

- Reports require date/order-based ranges.
- Searching is more important than insertion speed.

Hashed File

Best when:

- Transactions are retrieved using an exact key such as **TransactionID**.
- Very fast equality searches are required.

Recommendation: For a supermarket billing system with 1 million transactions per day, use Hashing for TransactionID-based retrieval, with a B+ Tree index on TransactionDate for reports and range queries.

Conclusion: A combined Hash + B+ Tree indexing strategy gives the best overall performance for high-volume billing transactions.